

연결 리스트 수업 정리 (단순/원형 연결 리스트 및 응용)

1. 단순 연결 리스트 (Singly Linked List)

- 각 노드는 데이터와 다음 노드를 가리키는 포인터를 가짐
- 마지막 노드의 포인터는 **NULL**
- 헤더 노드는 리스트의 시작을 나타냄

예시:

```
[10 | • ]→[20 | • ]→[30 | NULL]
  ↑
헤더
```

삽입 방법

1. 헤더가 NULL인 경우:

- 새 노드를 생성하고, 헤더가 이를 가리키게 설정

```
NULL → [10 | NULL] (헤더가 10을 가리킴)
```

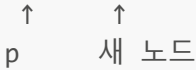
2. 리스트 맨 앞에 삽입 (Insert First):

- 새 노드를 생성하고, 새 노드의 링크가 기존 헤더를 가리키도록 설정
- 헤더를 새 노드로 갱신

```
[10 | • ]→[20 | • ]→[30 | NULL]
  ↑
헤더
↓ 삽입
[5 | • ]
결과:
[5 | • ]→[10 | • ]→[20 | • ]→[30 | NULL]
  ↑
헤더
```

3. 중간 삽입 (Insert Middle):

```
기존:
[10 | • ]→[30 | NULL]
      ↑
      p
삽입:
[10 | • ]→[20 | • ]→[30 | NULL]
```



삭제 방법

1. 첫 노드 삭제:

삭제 전:
[10 | •]→[20 | •]→[30 | NULL]
↑
헤더
삭제 후:
[20 | •]→[30 | NULL]
↑
헤더

2. 중간 노드 삭제:

삭제 전:
[10 | •]→[20 | •]→[30 | NULL]
 ↑ ↑
 prev target
삭제 후:
[10 | •]→[30 | NULL]

2. 원형 연결 리스트 (Circular Linked List)

- 마지막 노드의 포인터가 다시 헤더(첫 노드)를 가리킴
- 순환 구조이기 때문에 시작 지점 설정이 중요함

예시:

[10 | •]→[20 | •]→[30 | •]↘
 ↑ ↓
 └──────────────────┘
헤더: 마지막 노드를 가리킴
시작점: header->link = 첫 노드

삽입/삭제 방법

- **Insert First:**

```
[30 | • ]→[10 | • ]→[20 | • ]→ (30로 다시 연결)
↑
헤더
```

- **Insert Last:**

```
기존: [10 | • ]→[20 | • ]→[10 | • ] (순환)
신규 노드 30 삽입 → 헤더를 30으로 이동
결과:
[10 | • ]→[20 | • ]→[30 | • ]→[10 | • ]
                        ↑
                        헤더
```

3. 연결 리스트의 응용

(1) 탐색 연산

- 반복 또는 재귀로 리스트 순회하며 데이터 비교

```
헤더 → [10]→[20]→[30]→NULL
찾을 값: 20
→ 10? No → 20? Yes → return node
```

(2) 병합 연산

- 두 리스트를 하나로 연결

```
List1: [10 | • ]→[20 | • ]→[30 | NULL]
List2: [40 | • ]→[50 | NULL]
병합 후:
[10 | • ]→[20 | • ]→[30 | • ]→[40 | • ]→[50 | NULL]
```

(3) 다항식 덧셈

- 각 노드는 (계수, 지수)
- 같은 지수면 계수 더함, 다르면 큰 지수 먼저

예시:

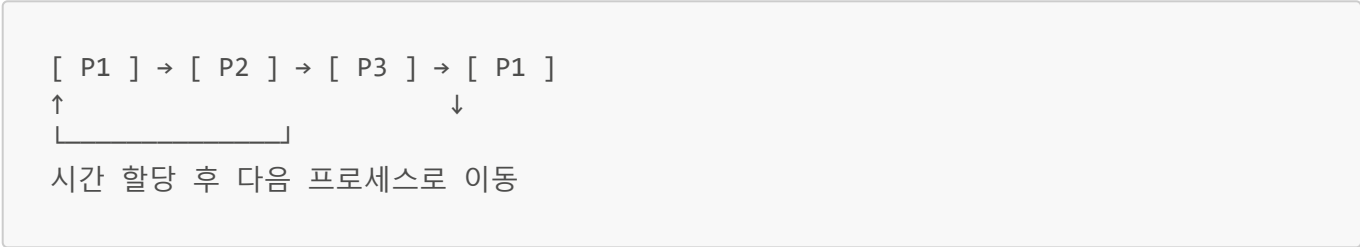
```
A(x): [3,2]→[2,1]→[1,0]
B(x): [2,2]→[1,1]
합:    [5,2]→[3,1]→[1,0]
```

```
struct PolyNode {
    int coef; // 계수
    int exp;  // 지수
    struct PolyNode* link;
};
```

4. 운영체제에서의 응용

- **Round-Robin** 스케줄링
- 원형 리스트로 각 프로세스 순환 처리

예시:



실습 예시 정리

Insert First 예시

입력 순서: 30, 20, 10
결과 리스트:
[10 | •]→[20 | •]→[30 | NULL]

Merge 예시

List1: [30]→[20]→[10]
List2: [50]→[40]
병합: [30]→[20]→[10]→[50]→[40]

다항식 예시

$A(x) = 3x^2 + 2x + 1$
 $B(x) = 2x^2 + x$
 $C(x) = 5x^2 + 3x + 1$

실습 과제

- 단순 연결 리스트 구현 (Insert/Delete/Search)
- 원형 연결 리스트 구현 및 응용 (스케줄링 시뮬레이션)
- 다항식 계산 프로그램 구현 (덧셈 등)

실습을 통해 구조 및 포인터 연산 원리를 이해하는 것이 중요합니다.